

Agent Mesh SRE — User Guide v3.0

Autonomous Monitor · MRAL Loop · Real-time Detection | June 2026

1. Introduction

Agent Mesh SRE v3.0 demonstrates fully autonomous AI-driven Kafka SRE operations. The Monitor agent now watches the cluster continuously and fires all 11 failure scenarios without any button clicks — pausing only when a scenario requires human approval before taking a potentially destructive action.

This guide covers the v3.0 fork at [surajcsibm/agent-mesh-sre-real](https://github.com/surajcsibm/agent-mesh-sre-real). The application operates in two modes: MOCK (simulation only, no Kafka cluster needed) and REAL (connects to Aiven for Apache Kafka).

2. What Is Agent Mesh SRE?

Agent	Role	Colour	New in v3.0
INTAKE	Ingests Kafka metric events	Emerald green	—
MONITOR	Autonomous MRAL loop + anomaly detection	Violet	Now auto-fires all 11 scenarios
WRITER	Generates structured incident reports	Cyan	—
NOTIFY	Sends email + notification popup	Orange	Fires for every autonomous scenario

3. What Happens Without Clicking Anything

This is the key new behaviour in v3.0. Once you open the app and log in:

Time after load	What you see
0s	Dashboard loads. TopBar shows cyan Monitor pill with cycle count.
30s	First poll cycle completes. ScenarioPanel MonitorStatusCard shows 'polling'.
35s	FIRST AUTONOMOUS SCENARIO fires. Canvas animates — no click needed.
35s (if approval-gated)	Policy Gate modal appears — read the reasoning and click Approve or Reject.

35s	Notification popup appears in right panel. Email sent to NOTIFICATION_EMAIL.
65s (if gated + approved)	Amber ring glows on scenario button. [POLL] entry in Audit Log.
125s	Broker state heals. Topic auto-heal MRAL fires for affected topics.
125–185s	Second autonomous scenario fires. Cycle repeats every 90–150 seconds.

4. Dashboard Layout

The dashboard has three vertical regions:

Left sidebar

- Monitor Status Card (top) — shows poll loop cycle count, running state, and detected scenarios from the last cycle
- Common Scenarios (4) — always visible with detection rings
- More Scenarios (6) — expanded by default in v3.0 — all 6 visible without clicking accordion
- Kafka Topics panel — shows topic health with ⚡ Monitor alert chips when anomalies are detected

Centre canvas

Agent nodes with live MRAL animation. Sub-agent circles appear during active phases. The canvas fires autonomously — you will see it animate without clicking any button.

Right sidebar

- Audit Log — ▶ [POLL] entries (cyan border) for every autonomous detection
- Notifications — popup for every scenario completion
- Approvals — pending and past approval decisions

5. Detection Rings on Scenario Buttons

Every scenario button now shows a visual state ring driven by the Monitor's autonomous detection:

Ring	Colour	Meaning
• detected	Cyan pulse	Monitor has detected conditions for this scenario. Within cooldown — not re-firing yet.

⚡ auto-fired	Amber glow	Monitor autonomously triggered this scenario. MRAL animation played with no button press.
⊘ suppressed	Purple ring	Monitor detected conditions but made an intelligent decision NOT to act (scenarios 04, 11).
No ring	—	No detection in the last 2-minute window. Broker state is healthy for this scenario.

Rings persist for 2 minutes after detection so you can see them even between 30-second poll cycles.

6. Running a Scenario Manually

You can still click any scenario button to trigger it immediately, bypassing the autonomous schedule:

- Click a scenario card — animation starts instantly
- If approval-gated: Policy Gate modal appears ~6 seconds into the scenario
- Approve to continue or Reject to abort — both outcomes are logged
- Scenario End Modal shows full summary — saved to history bar
- The Monitor poll loop detects the manually-triggered anomaly on the next cycle and logs a [POLL] detection entry

7. Approval Gates

4 scenarios pause for human approval. These require a human to review the Monitor's reasoning before any infrastructure mutation is executed:

Scenario	Action Requiring Approval	What Happens if Approved
Consumer Lag Spike	kafka.scaleConsumers — scale consumer group	Monitor scales consumers. Lag returns to 0. Topic heals after 90s.
Share Group Rebalance	kafka.checkpointShareGroup — checkpoint offsets	Monitor checkpoints. Share group stabilises.
Schema Registry Mismatch	kafka.updateSchemaCompatibility — mutate schema registry	Schema compatibility updated. DLQ drain begins.
Under-Replicated Partitions	Partition reassignment — mutate topic leadership	ISR restored. Under-replication cleared.

Rejected actions are fully audited. The Monitor logs the rejection, records a lesson, and does not mutate any infrastructure. The 5-minute cooldown still applies after a rejection, so the same scenario will not fire again for 5 minutes.

8. Kafka Topics Panel

Auto-create on scenario trigger

When any scenario fires (manually or autonomously), its relevant Kafka topics are automatically created in the Topics panel — or updated with scenario-specific lag and health values if they already exist.

Monitor alert chips

When the Monitor poll loop detects a scenario whose conditions relate to a specific topic's health (e.g. high lag, under-replication), that topic row shows small cyan ⚡ scenario-id chips below the lag metric. These appear within 5 seconds of detection and clear when the condition resolves.

Autonomous topic healing

90 seconds after each scenario's anomaly is injected, the Monitor triggers a dedicated MRAL healing cycle for affected topics. You will see the canvas animate as the Monitor heals each topic without any button press. The topic's status returns to Healthy after the cycle completes.

9. Audit Log — [POLL] Entries

The Audit Log in the right panel distinguishes autonomous detections from manual triggers:

- [POLL] Autonomous trigger — controller-failover (97%) — cyan left border, autopoll badge
- [POLL] Suppress — benign-rebalance — purple border — Monitor decided not to act
- [ANOMALY] Monitor detected: lag-spike — amber border — anomaly injected, waiting for poll cycle
- simulate-controller-failover — blue border — manual button click

Filter: look for entries with [POLL] prefix and cyan/autopoll styling to see Monitor autonomous activity.

10. Notifications & Email

The NOTIFY agent fires at the end of every scenario (autonomous or manual):

- Notification popup appears in the Notifications tab (right panel)
- Email sent to NOTIFICATION_EMAIL (if configured)
- Email includes: scenario name, approval decision, lag delta, reasoning, action taken, lesson recorded, full live events table

If SMTP is not configured, the notification popup still appears — only the email is skipped. The Scenario End Modal shows an email error notice.

11. Scenarios Reference

Common Scenarios (always visible)

Scenario	Gate	Topics Auto-Created	Auto-fires after
Consumer Lag Spike	Approval	ops.kafka.metrics.v1 (critical), ops.requests.v1	~35s
KRaft Controller Failover	Auto	ops.incidents.v1, ops.kafka.metrics.v1	~125s
Share Group Rebalance	Approval	share.group.events.v1 (critical), ops.kafka.metrics.v1	~215s
False-Positive Suppression	SUPPRESS	ops.kafka.metrics.v1 (degraded)	~305s

More Scenarios (6 — expanded by default)

Scenario	Gate	Topics Auto-Created	Auto-fires after
Schema Registry Mismatch	Approval	schema.registry.events.v1 (degraded)	~395s
Broker Disk Saturation	Auto	ops.broker.disk.v1, audit.payment.events.v1 (critical)	~485s
Under-Replicated Partitions	Approval	payments.transactions.v1 (degraded), payments.settlements.v1 (critical)	~575s
Producer Timeout Storm	Auto	payments.transactions.v1 (critical)	~665s
Consumer Session Timeout	Auto	invoices.created.v1 (critical)	~755s
Log Compaction Lag	Auto	ops.lessons.v1 (degraded), audit.payment.events.v1 (critical)	~845s

All 11 scenarios cycle in order. The full cycle completes in approximately 16–28 minutes (randomised 90–150s intervals). The cycle then repeats from the beginning.

12. Monitor Status Indicators

Indicator	Location	What it shows
Cyan pulse pill	TopBar (top right of cluster status bar)	Poll loop running, cycle count, last detected count
MonitorStatusCard	Top of left scenario panel	Poll state, cycle count, last poll timestamp, detected scenarios with confidence
Detection rings	On each scenario button	cyan/amber/purple depending on detection state
[POLL] Audit entries	Right panel Audit Log	Every detection event with scenario ID, confidence, cause
 Topic chips	Topic rows in Topics panel	Monitor alert when poll detects scenario affecting that topic
/api/mesh/poll	Browser tab (debug)	Full JSON: running, cycleCount, detectedThisCycle, latestSnapshot

13. Deploying to Vercel

After all patches are applied:

- `git add -A && git commit -m 'feat: autonomous Monitor v3.0' && git push origin main`
- Vercel auto-deploys on push (~2 minute build)
- Set all env vars in Vercel project Settings → Environment Variables
- Add NOTIFICATION_EMAIL for email alerts on every autonomous scenario

Vercel note on poll loop: The polling loop runs within the 60-second SSE function lifetime. It gets up to 2 poll cycles per connection window. When the client auto-reconnects (every 60s), a new SSE function starts fresh — detection history resets. For a conference demo, open the app and keep a browser tab active; the loop runs continuously while the tab is open.

14. Troubleshooting

Issue	Cause	Fix
No autonomous MRAL after 60s	Poll loop not started	Open dashboard — SSE connects and starts both loops. Check terminal for [AnomalySim] Started.
/api/mesh/poll shows running:false	SSE not connected	Open main dashboard first at localhost:3002, then check the poll endpoint.
Detection rings not appearing	recentDetections buffer not populated	Wait 30s after scenario fires. Rings persist 2 min. Check detectedThisCycle in poll endpoint.

Approval gate not appearing	useMeshStream missing approval-new handler	Apply patch-14-approval-gate.py
Email not sent	SMTP not configured or NOTIFICATION_EMAIL missing	Add NOTIFICATION_EMAIL to .env.local. Use Gmail App Password for SMTP_PASS.
Topic chips not showing	patch-8 not applied	grep monDet Dashboard.tsx — if missing, re-run patch-8-full-detection-ui.py
Port 3002 already in use	Old dev server still running	lsof -ti :3002 xargs kill -9 && npm run dev
Webpack 404 on page load	Stale .next cache	rm -rf .next && npm run dev
Build fails on type error	TypeScript mismatch	npm run build 2>&1 grep 'Type error' to find cause